



درس برنامه نویسی همروند
نیم سال دوم ۱۴۰۴-۱۴۰۵
استاد: مهدی موحیدیان مقدم
دستیار آموزشی: سبا سلطانی فیروز

تمرین سری چهارم

- پرسش‌های خود را می‌توانید در گروه متصل به کانال تلگرامی درس (@concurrent4042) و در صورت عدم دسترسی به اینترنت بین‌الملل در گروه بله^۱، مطرح کنید.
- پاسخ‌های خود را در سامانه LMS ارسال فرمایید.
- نوشتن مشخصات خود را فراموش نفرمایید.

۱ مقدمه

در بسیاری از مسائل علمی و مهندسی، مقدار یک تابع فقط در تعدادی نقطه مشخص در اختیار ما قرار دارد. برای مثال، ممکن است مقدار دما، فشار، سرعت، غلظت یک ماده، یا خروجی یک آزمایش فقط در چند نقطه اندازه‌گیری شده باشد. در چنین شرایطی، اگر بخواهیم مقدار تابع را در نقطه‌ای بین نقاط معلوم تخمین بزنیم، از روش‌های درونیابی استفاده می‌کنیم. یکی از روش‌های کلاسیک درونیابی، روش درونیابی لاگرانژ است. در این روش، با استفاده از چند نقطه معلوم، یک چندجمله‌ای ساخته می‌شود که از تمام نقاط داده عبور می‌کند. سپس از این چندجمله‌ای برای تخمین مقدار تابع در نقاط جدید استفاده می‌شود. در این پروژه، هدف پیاده‌سازی سریال و موازی درونیابی لاگرانژ است. نسخه سریال روی CPU اجرا می‌شود و نسخه موازی باید با استفاده از CUDA روی GPU پیاده‌سازی شود. هدف اصلی پروژه، تمرین مفاهیم برنامه‌نویسی موازی روی GPU است؛ بنابراین تمرکز اصلی باید بر نحوه تقسیم محاسبات بین threadها، مدیریت حافظه GPU، اجرای kernel، انتقال داده بین CPU و GPU، و تحلیل زمان اجرا باشد.

۲ شرح مسئله

تابع زیر را در بازه $[0, \pi]$ در نظر بگیرید:

$$f(x) = \sin(x) \quad (۱)$$

فرض کنید مقدار این تابع فقط در تعدادی نقطه مشخص در اختیار ما قرار دارد. این نقاط را با نمادهای زیر نمایش می‌دهیم:

$$(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1}) \quad (۲)$$

که در آن:

^۱<https://ble.ir/join/GriwxnAhmG>

$$y_i = f(x_i) \quad (۳)$$

نقاط x_i را به صورت یکنواخت روی بازه $[0, \pi]$ انتخاب کنید:

$$x_i = a + ih, \quad i = 0, 1, \dots, n-1 \quad (۴)$$

که در آن:

$$a = 0, \quad b = \pi \quad (۵)$$

و:

$$h = \frac{b-a}{n-1} \quad (۶)$$

هدف این پروژه آن است که با استفاده از نقاط معلوم بالا، مقدار تابع را در تعداد زیادی نقطه جدید تقریب بزنیم. این نقاط جدید را نقاط پرس و جو یا query points می نامیم و با نمادهای زیر نشان می دهیم:

$$q_0, q_1, \dots, q_{m-1} \quad (۷)$$

برای هر نقطه q_j باید مقدار تقریبی تابع با استفاده از درونیابی لاگرانژ محاسبه شود.

۳ روش درونیابی لاگرانژ

در روش درونیابی لاگرانژ، با استفاده از n نقطه معلوم، یک چندجمله ای $P(x)$ ساخته می شود که از تمام نقاط داده عبور می کند. یعنی برای هر نقطه داده داریم:

$$P(x_i) = y_i, \quad i = 0, 1, \dots, n-1 \quad (۸)$$

چندجمله ای لاگرانژ به صورت زیر تعریف می شود:

$$P(x) = \sum_{i=0}^{n-1} y_i L_i(x) \quad (۹)$$

در این رابطه، $L_i(x)$ پایه لاگرانژ نام دارد و به صورت زیر محاسبه می شود:

$$L_i(x) = \prod_{\substack{k=0 \\ k \neq i}}^{n-1} \frac{x - x_k}{x_i - x_k} \quad (۱۰)$$

بنابراین برای محاسبه مقدار تقریبی تابع در نقطه q_j ، باید مقدار زیر محاسبه شود:

$$P(q_j) = \sum_{i=0}^{n-1} y_i L_i(q_j) \quad (11)$$

به زبان ساده، برای هر نقطه پرس و جو q_j ، باید سهم تمام نقاط داده محاسبه شود و سپس این سهم ها با هم جمع شوند تا مقدار نهایی درونیابی به دست آید.

۴ ایده موازی سازی با CUDA

نکته مهم در این پروژه آن است که محاسبه مقدار درونیابی برای هر نقطه پرس و جو مستقل از سایر نقاط پرس و جو است. یعنی محاسبه $P(q_0)$ به نتیجه $P(q_1)$ وابسته نیست و محاسبه $P(q_1)$ نیز به نتیجه $P(q_2)$ وابسته نیست. به همین دلیل، این مسئله برای موازی سازی روی GPU مناسب است. در نسخه CUDA، هر thread می تواند مسئول محاسبه مقدار درونیابی برای یک نقطه پرس و جو باشد. به عبارت دیگر، اگر m نقطه پرس و جو داشته باشیم، می توانیم محاسبه این m مقدار را بین تعداد زیادی thread تقسیم کنیم.

در گزارش باید توضیح دهید که هر thread مسئول چه کاری است و چگونه اندیس نقطه پرس و جو مربوط به هر thread تعیین می شود. همچنین باید توضیح دهید که داده های مربوط به نقاط معلوم، نقاط پرس و جو و خروجی ها چگونه در حافظه GPU قرار گرفته اند.

۵ پارامترهای پیشنهادی پروژه

برای تعداد نقاط داده، مقدارهای زیر را بررسی کنید:

$$n = 8, 16, 32, 64 \quad (12)$$

برای تعداد نقاط پرس و جو، مقدارهای زیر را بررسی کنید:

$$m = 10^4, 10^5, 10^6 \quad (13)$$

در صورت امکان و در صورتی که سیستم شما حافظه و توان پردازشی کافی دارد، می توانید مقدارهای بزرگ تر برای m را نیز بررسی کنید.

توجه کنید که در این پروژه، تعداد نقاط داده n معمولاً کوچک یا متوسط در نظر گرفته می شود، اما تعداد نقاط پرس و جو m می تواند بسیار بزرگ باشد. دلیل این انتخاب آن است که هدف اصلی پروژه بررسی موازی سازی محاسبات برای تعداد زیادی نقطه مستقل است.

۶ پیاده سازی سریال روی CPU

در مرحله اول، باید نسخه سریال درونیابی لاگرانژ روی CPU پیاده سازی شود. در این نسخه، مقدار درونیابی برای تمام نقاط پرس و جو به صورت ترتیبی محاسبه می شود. نسخه سریال باید شامل موارد زیر باشد:

- تولید نقاط داده x_i روی بازه $[0, \pi]$

- محاسبه مقدار $y_i = \sin(x_i)$
- تولید نقاط پرس وجو q_j روی بازه $[0, \pi]$
- محاسبه مقدار درونیابی $P(q_j)$ برای تمام نقاط پرس وجو
- محاسبه خطا نسبت به مقدار واقعی $\sin(q_j)$
- اندازه گیری زمان اجرای نسخه سریال

این نسخه به عنوان مبنای مقایسه با نسخه CUDA استفاده می شود. کد سریال مرجع در جلسه معرفی پروژه توضیح داده خواهد شد.

۷ پیاده سازی موازی با CUDA

در مرحله دوم، باید نسخه موازی برنامه با استفاده از C/C++ CUDA پیاده سازی شود. در این نسخه، محاسبه مقدارهای درونیابی برای نقاط پرس وجو باید روی GPU انجام شود. در پیاده سازی CUDA باید موارد زیر انجام شود:

- اختصاص حافظه برای آرایه های نقاط داده روی GPU
- اختصاص حافظه برای آرایه نقاط پرس وجو روی GPU
- اختصاص حافظه برای آرایه خروجی روی GPU
- انتقال داده های اولیه از CPU به GPU
- اجرای kernel برای محاسبه مقدارهای درونیابی
- انتقال خروجی نهایی از GPU به CPU
- آزادسازی حافظه های اختصاص داده شده

در گزارش باید توضیح دهید که اندازه block و grid را چگونه انتخاب کرده اید. همچنین باید مشخص کنید هر thread دقیقاً مسئول محاسبه کدام مقدار خروجی است.

۸ بررسی صحت پیاده سازی

برای بررسی درستی برنامه، خروجی نسخه سریال و نسخه CUDA باید با یکدیگر مقایسه شوند. چون هر دو نسخه از یک روش عددی استفاده می کنند، انتظار می رود خروجی آن ها بسیار نزدیک به هم باشد. برای مقایسه نسخه CPU و نسخه GPU می توانید از خطای بیشینه زیر استفاده کنید:

$$Error_{CPU-GPU} = \max_j |P_{CPU}(q_j) - P_{GPU}(q_j)| \quad (۱۴)$$

همچنین برای بررسی دقت روش درونیابی نسبت به تابع اصلی، خطای زیر را محاسبه کنید:

$$Error_{max} = \max_j |P(q_j) - \sin(q_j)| \quad (۱۵)$$

می‌توانید خطای میانگین را نیز گزارش کنید:

$$Error_{avg} = \frac{1}{m} \sum_{j=0}^{m-1} |P(q_j) - \sin(q_j)| \quad (۱۶)$$

در گزارش باید توضیح دهید که با افزایش تعداد نقاط داده n ، خطای درونیابی چه تغییری می‌کند.

۹ آزمایش‌های کارایی

برنامه را برای اندازه‌های مختلف n و m اجرا کنید. برای هر حالت، زمان اجرای نسخه CPU و نسخه CUDA را اندازه‌گیری کنید. سپس مقدار speedup را به صورت زیر محاسبه کنید:

$$Speedup = \frac{T_{CPU}}{T_{GPU}} \quad (۱۷)$$

که در آن T_{CPU} زمان اجرای نسخه سریال روی CPU و T_{GPU} زمان اجرای نسخه موازی روی GPU است. در گزارش باید تحلیل کنید که در چه حالتی استفاده از GPU باعث بهبود زمان اجرا شده است. همچنین توضیح دهید چرا برای تعداد کم نقاط پرس‌وجو، ممکن است نسخه CUDA نسبت به نسخه CPU برتری محسوسی نداشته باشد. عواملی مانند سربار انتقال داده بین CPU و GPU، سربار اجرای kernel، و اندازه مسئله باید در تحلیل در نظر گرفته شوند.

۱۰ بررسی اثر اندازه Block

یکی از بخش‌های مهم پروژه، بررسی اثر اندازه block بر زمان اجرای نسخه CUDA است. برنامه را با چند مقدار مختلف برای تعداد thread در هر block اجرا کنید. به عنوان نمونه، مقدارهای زیر را بررسی کنید:

$$64, 128, 256, 512 \quad (۱۸)$$

برای هر حالت، زمان اجرای نسخه CUDA را گزارش کنید. سپس توضیح دهید که تغییر اندازه block چه اثری بر عملکرد برنامه داشته است و چرا انتخاب اندازه مناسب برای block در برنامه‌های CUDA اهمیت دارد.

۱۱ خروجی‌های مورد انتظار

دانشجو باید کد کامل پروژه و یک گزارش تحلیلی ارائه دهد. گزارش باید شامل موارد زیر باشد:

- توضیح پیاده‌سازی سریال روی CPU
- توضیح پیاده‌سازی موازی با CUDA
- توضیح نحوه نگاشت نقاط پرس‌وجو به threadهای CUDA
- توضیح نحوه مدیریت حافظه در GPU
- جدول مقایسه زمان اجرای CPU و GPU

- محاسبه و تحلیل speedup
- بررسی اثر اندازه block
- محاسبه خطای نسخه CPU و نسخه GPU
- محاسبه خطای درونیابی نسبت به تابع اصلی
- رسم نمودار تابع اصلی و تابع درونیابی شده برای چند مقدار مختلف n

وجود حداقل سه جدول در گزارش الزامی است:

- جدول خطای درونیابی برای مقادیرهای مختلف n
- جدول زمان اجرای CPU و GPU برای مقادیرهای مختلف m
- جدول اثر اندازه block بر زمان اجرای نسخه CUDA

۱۲ نکات مربوط به ابزار پیاده‌سازی

پیاده‌سازی اصلی پروژه باید با استفاده از یکی از روش‌های مبتنی بر CUDA انجام شود. هدف پروژه این است که محاسبات اصلی درونیابی روی GPU انجام شود و دانشجو با مفاهیمی مانند thread, block, grid, مدیریت حافظه GPU و انتقال داده بین CPU و GPU درگیر شود.

استفاده از multiprocessing, Python threading, MPI, OpenMP یا کتابخانه‌های آماده درونیابی برای بخش اصلی پروژه مجاز نیست؛ زیرا این روش‌ها هدف اصلی تمرین، یعنی پیاده‌سازی محاسبات روی GPU با CUDA، را پوشش نمی‌دهند. در صورتی که از محیط‌هایی مانند Visual Studio، Google Colab، Jupyter Notebook یا سایر محیط‌های اجرایی استفاده می‌کنید، روش اجرای برنامه و تنظیمات لازم را در گزارش توضیح دهید.

۱۳ توضیحات تکمیلی

در این پروژه، هدف اصلی یادگیری درونیابی لاگرانژ نیست، بلکه هدف اصلی پیاده‌سازی یک محاسبه عددی مستقل و پرتکرار روی GPU است. بنابراین، گزارش باید نشان دهد که دانشجو توانسته است محاسبات نسخه سریال را به درستی به نسخه موازی CUDA منتقل کند.

کیفیت تحلیل نتایج اهمیت زیادی دارد. سریع‌تر بودن نسخه CUDA برای همه اندازه‌های مسئله تضمین شده نیست. اگر تعداد نقاط پرس‌وجو کم باشد، سربار انتقال داده و اجرای kernel ممکن است باعث شود نسخه CPU سریع‌تر یا نزدیک به نسخه GPU باشد. در مقابل، برای تعداد زیاد نقاط پرس‌وجو، انتظار می‌رود توان موازی GPU بهتر دیده شود. گزارش باید مستقل و قابل فهم باشد. یعنی خواننده باید بتواند از روی گزارش متوجه شود مسئله چه بوده، روش لاگرانژ چگونه استفاده شده، برنامه چگونه اجرا شده، چه پارامترهایی انتخاب شده، و نتایج به دست آمده چه معنایی دارند.

با آرزوی موفقیت